MASTER PROMPT — ADAPTIVE FOCUS MUSIC SYSTEM

Create a desktop software system called “Adaptive Cognitive Soundscape” that dynamically adjusts ambient focus music based on the user’s current work context and cognitive state.

The system is NOT a traditional music player. It is a closed-loop adaptive auditory environment designed to stabilize attention, reduce distraction, and support sustained cognitive work.

The software should continuously monitor user activity on the computer and gradually adapt music in real time without abrupt transitions.

CORE DESIGN PHILOSOPHY

The system should behave like an intelligent adaptive soundtrack engine similar to dynamic game music systems.

The software should:

- detect work context
- estimate cognitive state
- select or blend suitable music layers
- smoothly transition between audio environments
- avoid distracting the user

The music should NEVER:

- become emotionally overwhelming
- aggressively demand attention
- abruptly change
- sound like EDM or cinematic trailer music

The auditory environment should feel:

- immersive
- slow-paced
- deliberate
- restrained
- subtly tense
- cognitively supportive

The overall atmosphere should resemble:

- engineering advanced technology late at night
- collaborative design work under calm pressure
- scientific discovery

- quiet focused creativity
- productive concentration with restrained momentum

CORE SYSTEM ARCHITECTURE

The project consists of five major subsystems:

1. User Activity Monitor
2. Cognitive State Estimator
3. Context Classifier
4. Adaptive Music Engine
5. Transition Controller

---

1. USER ACTIVITY MONITOR

The software should continuously monitor:

- active application
- browser tab usage
- keyboard activity
- typing cadence
- mouse movement
- window switching frequency
- idle duration
- task duration
- focus duration

Suggested implementation:

- Python backend
- OS-level hooks
- activity logging
- event-driven architecture

Possible libraries:

- pywin32
- pynput
- psutil
- pyautogui

---

2. CONTEXT CLASSIFIER

The software should classify work contexts.

Example contexts:

Programming / Engineering

Detected via:

- VSCode
- JetBrains IDEs
- terminals
- GitHub
- documentation tabs

Deadline / Team Workflow

Detected via:

- rapid but controlled productivity
- multiple productivity apps
- communication apps
- collaborative software

Reading / Writing

Detected via:

- PDFs
- browsers
- word processors
- note-taking apps

Scientific / Mathematical Work

Detected via:

- LaTeX editors
- research papers
- symbolic computation software
- simulation tools

Creative Design Work

Detected via:

- Figma
- Photoshop
- Blender
- design software

Distraction State

Detected via:

- social media tabs
- gaming
- rapid context switching
- short attention intervals
- repeated app switching

The classifier should initially use rule-based heuristics.

The architecture should later support machine learning classifiers.

---

3. COGNITIVE STATE ESTIMATOR

The system should estimate:

- deep focus
- mild distraction
- overstimulation
- fatigue
- calm productivity
- sustained flow state

The estimator should infer state using:

- typing rhythm consistency
- app switching entropy
- mouse movement smoothness
- task persistence
- idle pauses
- time-on-task

The software should avoid binary “focused/distracted” logic.

Use probabilistic scoring instead.

Example: focus_score = 0.0–1.0

---

4. ADAPTIVE MUSIC ENGINE

The system should use:

- pre-generated looping music environments
- synchronized layered stems
- adaptive audio transitions

DO NOT generate entirely new songs in real time.

The system should instead:

- blend
- crossfade
- add/remove layers
- apply filters
- modify intensity gradually

Suggested technologies:

- FMOD Studio
- Wwise
- Max/MSP
- SuperCollider

FMOD is preferred for the MVP.

MUSIC STRUCTURE

Each environment should contain synchronized layers:

Base Ambient Layer

- sustained textures
- drones
- atmospheric pads

Rhythm Layer

- soft restrained pulse
- heavily muffled percussion
- low rhythmic density

Harmonic Layer

- slow chord movement
- subtle unresolved harmony

Accent Layer

- sparse detail textures
- subtle mechanical sounds
- low-volume events

GLOBAL AUDIO RULES

The music must:

- feel genuinely slow-paced
- avoid psychological double-tempo perception
- use quarter-note pulse
- contain large amounts of negative space
- evolve gradually
- avoid emotional climax

Percussion must:

- be heavily muffled
- deeply buried in the mix
- low-volume
- low-transient

Avoid:

- rapid hi-hats
- EDM energy
- cinematic trailer drums
- aggressive basslines
- bright lead synths
- excessive melodic hooks
- dramatic transitions

---

5. TRANSITION CONTROLLER

Transitions must:

- be gradual
- preserve immersion
- avoid disrupting flow state

Transition methods:

- crossfades
- filter interpolation
- layer fading
- harmonic compatibility checks

The system should:

- transition slowly during deep focus
- transition faster during distraction recovery

The system should include:

- cooldown periods
- hysteresis
- transition thresholds

to prevent constant oscillation.

MUSIC ADAPTATION LOGIC

Understimulation / Boredom

Increase:

- subtle pulse
- brightness slightly
- motion gently

Maintain restraint.

Overstimulation

Reduce:

- rhythmic density
- harmonic complexity
- spectral brightness

Increase:

- atmospheric continuity

Deep Focus

Minimize changes.

Preserve continuity.

Fatigue

Introduce:

- slightly warmer textures
- softer pacing

- restorative ambience

REQUIRED SOFTWARE FEATURES

Adaptive Layer Mixing

Real-time control of:

- volume
- filtering
- density
- texture layers

Context Persistence

Avoid rapid switching between contexts.

Seamless Looping

All tracks must loop transparently.

Real-Time Parameter System

Expose parameters:

- focus intensity
- energy
- complexity
- warmth
- tension

UI DESIGN

The UI should be:

- minimal
- non-distracting
- futuristic
- dark-themed

Display:

- current detected context
- estimated focus state
- active soundtrack profile

Include:

- manual override
- sensitivity controls
- privacy settings

PRIVACY REQUIREMENTS

The system must:

- process locally when possible
- avoid storing sensitive content
- avoid transmitting user data externally

The system should monitor:

- metadata NOT:

- actual text content

- passwords
- personal messages

RECOMMENDED TECH STACK

Backend:

- Python

Frontend:

- Electron or Qt

Audio:

- FMOD Studio API

ML / State Estimation:

- scikit-learn
- PyTorch later

Data Pipeline:

- asynchronous event processing

MVP IMPLEMENTATION PRIORITY

PHASE 1:

- activity monitoring
- rule-based context detection
- adaptive track switching
- smooth crossfades

PHASE 2:

- layered adaptive music
- probabilistic focus scoring
- personalized adaptation

PHASE 3:

- reinforcement learning
- predictive distraction detection
- individualized audio optimization

IMPORTANT DESIGN PRINCIPLE

The software should NOT maximize:

- stimulation
- productivity at all costs
- constant engagement

The software SHOULD optimize:

- attentional stability
- sustainable concentration
- reduced cognitive fragmentation
- long-session mental comfort

The result should resemble an intelligent adaptive soundtrack for deep cognitive work rather than a productivity gimmick.